



SOFTWARE DESIGN DOCUMENT

for

Software Risk Tracker

Release 1.0

Version 1.0 approved

Prepared by Emily, Reddy, Logan, Nick, Alli, Brandon, Hannah, Josh, and Shelby



Contents

List of Figures	4
List of Tables	6
1 INTRODUCTION	8
1.1 Purpose	8
1.2 Scope	8
1.3 Overview	9
1.4 References	10
1.5 Definitions and Acronyms	10
2 SYSTEM OVERVIEW	12
3 SYSTEM ARCHITECTURE	13
3.1 Architectural Design	13
3.2 Decomposition Description	16
3.2.1 Object-Oriented Decomposition (Class Structure) . .	16
3.2.2 Database Schema Decomposition	18
3.3 Design Rationale	23
4 DATA DESIGN	24
4.1 Data Description	24
4.2 Data Dictionary	24
5 COMPONENT DESIGN	26
5.1 Components	27
5.1.1 User Interface (UI)	36

5.1.2	Application Layer (API)	38
5.1.3	Authenticator	40
5.1.4	User Administration	42
5.1.5	Risk Management	44
5.1.6	Storage of Data	47
6	HUMAN INTERFACE DESIGN	49
6.1	Overview of User Interface	49
6.2	Screen Designs	49
7	REQUIREMENTS TRACEABILITY MATRIX	59

List of Figures

3.1	Subsystem interactions	15
3.2	Package Diagram	18
3.3	User Use Case	20
3.4	Admin Use Case	21
3.5	Database ERD	22
5.1	Overall Class Structuring	27
5.2	User Login/Navigation Control Flow	28
5.3	Add Risk Control Flow	29
5.3	Add Risk Control Flow (Continued)	30
5.3	Add Risk Control Flow (Continued)	31
5.4	User Administration Control Flow	32
5.4	User Administration Control Flow (Continued)	33
5.4	User Administration Control Flow (Continued)	34
5.5	User Interface Class Diagram	36
5.6	API Class Diagram	38
5.7	Authenticator Class Diagram	40
5.8	User Administration Diagram	42
5.9	Risk Management Diagram	44
5.10	Storage of Data Class Diagram	47
6.1	Risk Tracker Login	50
6.2	Risk Tracker Login Fail	51
6.3	Non-Admin User Dashboard	51
6.4	Create Project	52
6.5	Edit project	52

6.6	Add new risk to Matrix	53
6.7	edit current risk	53
6.8	Matrix display	54
6.9	Admin dashboard - select project	55
6.10	confirm delete Project	55
6.11	Add Editor	56
6.12	Remove Editor	56
6.13	Confirm remove editor	57
6.14	Create User	57
6.15	Delete User	58
6.16	Confirm delete user	58

List of Tables

1.1	Acronyms and their descriptions	11
7.1	Nonfunctional Requirements and their Corresponding Components	61

Revision History

Date	Description	Revised by
02/4/26	Initial Baseline	Risk Team

1 INTRODUCTION

1.1 Purpose

This software design document (SDD) describes the architecture and system design of the Software Risk Tracker, a project management web application. The Software Risk Tracker is designed to help users keep track of potential risks for a project. This document is intended for project managers, software engineers, testers, and anyone who collaborates with the software development team.

1.2 Scope

The software design document is for a base-level system that will work as a proof of concept for a tool that allows software teams to manage development risks. This Software Design is focused on the base level system and critical parts of the system, by describing the implementation details of the Software Risk Tracker web application. The critical parts include these major components: Risk Management I/O, User Authentication, Notification Handling, and Data Storage.

The goals of this document are as follows:

- 1: Outline the architectural design of the system.

- 2: Describe how the data will be processed, stored, and organized.
- 3: Describe the behavior and purpose of each component.
- 4: Detail the design of the user interface.

1.3 Overview

The Software Design Document is divided into 7 sections with various sub-sections. The sections of the Software Design Document are:

- 1 Introduction
- 2 System Overview
- 3 System Architecture
- 4 Data Design
- 5 Component Design
- 6 Human Interface Design
- 7 Requirements Traceability Matrix

1.4 References

- [1] ACCESS, L. Digital accessibility solutions and services. <https://www.levelaccess.com/homepage-ppc/>, 2025. Accessed: 2025-10-21.
- [2] OF ELECTRICAL, I., AND (IEEE), E. E. Ieee official website. <https://www.ieee.org/>, 2025. Accessed: 2025-10-21.
- [3] SOFTWARE ENGINEERING INSTITUTE, C. M. U. Sei cert c++ coding standard – rule 09: Object oriented programming (oop). <https://wiki.sei.cmu.edu/confluence/pages/viewpage.action?pageId=88046338>, 2025. Accessed: 2025-10-21.
- [4] (W3C), W. W. W. C. Web content accessibility guidelines (wcag) 2.1. <https://www.w3.org/TR/WCAG21/>, 2018. Accessed: 2025-10-21.

1.5 Definitions and Acronyms

Acronym	Description
UI	User Interface
API	Application Layer
AUM	Authentication and User Management
RM	Risk Management
SoD	Storage of Data

Table 1.1: Acronyms and their descriptions

2 SYSTEM OVERVIEW

The Software Risk Tracker is a web-based multi-user management system designed to help developers record, monitor, and analyze risks associated with their projects. The system supports creating and managing multiple software projects simultaneously and enables exporting the entire risk list in CSV format for reporting or auditing purposes.

The system follows a client server architecture and is developed using the C++ programming language and implemented with the Webtoolkit (Witty) framework. Users access the application through any web browser. The system supports multiple sessions, allowing different users to view and modify project risks in real time.

3 SYSTEM ARCHITECTURE

3.1 Architectural Design

The Software Risk Tracker is designed using an object-oriented approach with a model-view architecture: the system is divided into separate modules, each responsible for a specific task. This level of organization makes for an easy to build-upon, maintain, and understand product.

Major subsystems and their responsibilities include:

1: User Interface (UI) -

Allows users to view and interact with risks.

Shows dashboards, risk lists, and forms to create or edit risks . [4]

2: Application Layer (API) -

Handles requests from the UI.

Processes business rules such as risk storing and validation.

Sends or retrieves data from the database. [3]

3: Authentication -

Ensures that only authorized users can view or modify risks through credential verification. [1]

4: User Administration -

Manages creating and deleting user accounts.

Defines and assigns permissions for admin and regular users.

Implements inactivity timeouts. [1]

5: Risk Management (RM) -

Core logic for creating, updating, and tracking risks.

Maintains the status, priority, and history of risks. [3] [2]

Subsystem dependencies are shown in figure 3.1.

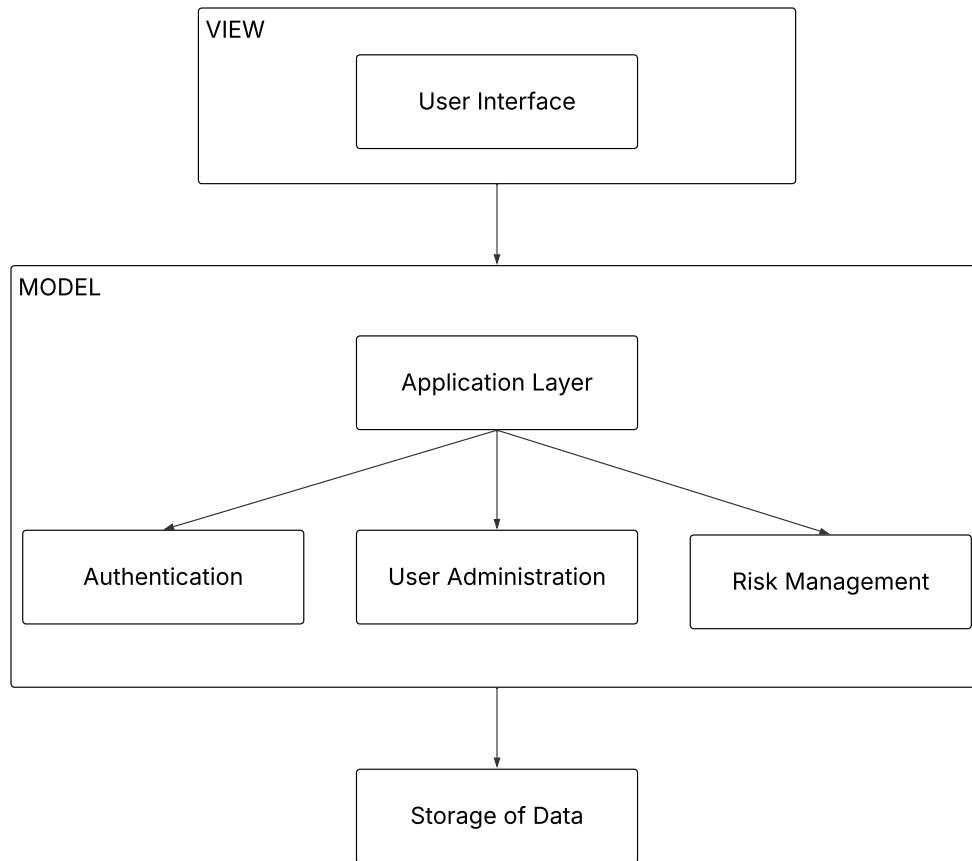


Figure 3.1: Subsystem interactions

6: Storage of Data (SoD) -

Includes risk tracker data and account data (usernames, passwords, etc).

These subsystems work together through these aspects:

- Users interact with the UI, which sends requests to the API layer.
- The API calls the Risk Management or User Administration or Authentication modules to process the request.
- Any data changes are saved in the database.

3.2 Decomposition Description

The system is broken down into smaller, easier-to-manage parts including the frontend, backend, and other modules:

This section details the internal decomposition of the subsystems identified in Section 3.1. It breaks down the architecture into specific C++ classes and database schemas used to implement the system.

3.2.1 Object-Oriented Decomposition (Class Structure)

The application logic is implemented in C++ using an Object-Oriented approach. Figure 3.2 illustrates how the packages interact. The primary classes and their responsibilities are defined below:

RiskTrackerApp

Inherits from `Wt::WApplication`. This is the main entry point for the web session. It initializes the UI and manages the life-cycle of the user's session.

UserManager

Handles authentication logic.

- **Methods:** `login(username, password)`, `logout()`, `createUser()`, `idleTimer()`

- **Collaborations:** Queries the `DatabaseManager` to verify credentials.

`RiskMatrix`

Responsible for the visual and interactive representation of the risk grid. It maps data objects to the user interface and handles the dynamic rendering of the color-coded table.

- **Methods:** `renderGrid(rows, cols)`, `assignCellColor(score)`, `handleCellClick(row, col)`
- **Attributes:** `int dimensions`, `std::vector<RiskItem> displayList`
- **Collaborations:** Receives data from `RiskManager` to populate the visual cells; provides the UI component to `RiskTrackerApp`.

`RiskManager`

Contains the business logic for calculating risk scores and managing risk lists.

- **Methods:** `addRisk(RiskItem)`, `updateRisk(id, RiskItem)`, `calculateSeverity(prob, impact)`
- **Collaborations:** Sends data to `DatabaseManager` for persistence; updates the UI models.

`DatabaseManager`

A single class responsible for all SQLite interactions.

- **Methods:** `executeQuery(sql)`, `openDB()`, `closeDB()`
- **Attributes:** Holds the `sqlite3*` connection handle.

`RiskItem (Data Object)`

A plain old data (POD) structure or class representing a single risk.

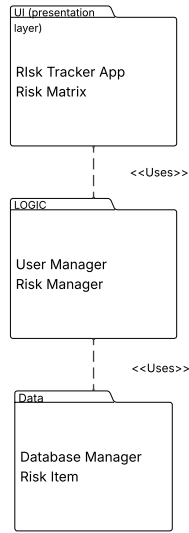


Figure 3.2: Package Diagram

- **Attributes:** int id, std::string title, int probability, int impact.

3.2.2 Database Schema Decomposition

The system uses a relational database (SQLite). The decomposition of data storage is defined by the following schema tables:

Table Name	Column	Type
User	UserID (PK)	INTEGER
	Name	VARCHAR(30)
	Email	VARCHAR(30)
	PasswordHash	TEXT
Role	UserID (PK)	INTEGER
	RoleTitle (PK)	VARCHAR(15)

Table Name	Column	Type
Project	ProjectID (PK)	INTEGER
	Name	VARCHAR(50)
	ImpactCount	INTEGER
	ProbCount	INTEGER
	Created	TIMESTAMP
	Modified	TIMESTAMP
Access	UserID (FK)	INTEGER
	ProjectID (FK)	INTEGER
ScaleLabels	ScaleID (PK)	INTEGER
	ProjectID (FK)	INTEGER
	Type	CHARACTER(1)
	Name	VARCHAR(20)
	Level	INTEGER
Risks	RiskID (PK)	INTEGER
	ProjectID (FK)	INTEGER
	Name	VARCHAR(30)
	Desc	VARCHAR(100)
	Prob	INTEGER
	Impact	INTEGER
AuditLog	LogID (PK)	INTEGER
	Timestamp	TIMESTAMP
	Username	VARCHAR(50)
	EventType	VARCHAR(15)
	EntityType	VARCHAR(15)
	Details	TEXT

The ERD diagram in figure 3.5 illustrates the relationships between the data tables. Figures 3.3 and 3.4 show two use cases, one for a regular user and one for an admin user.

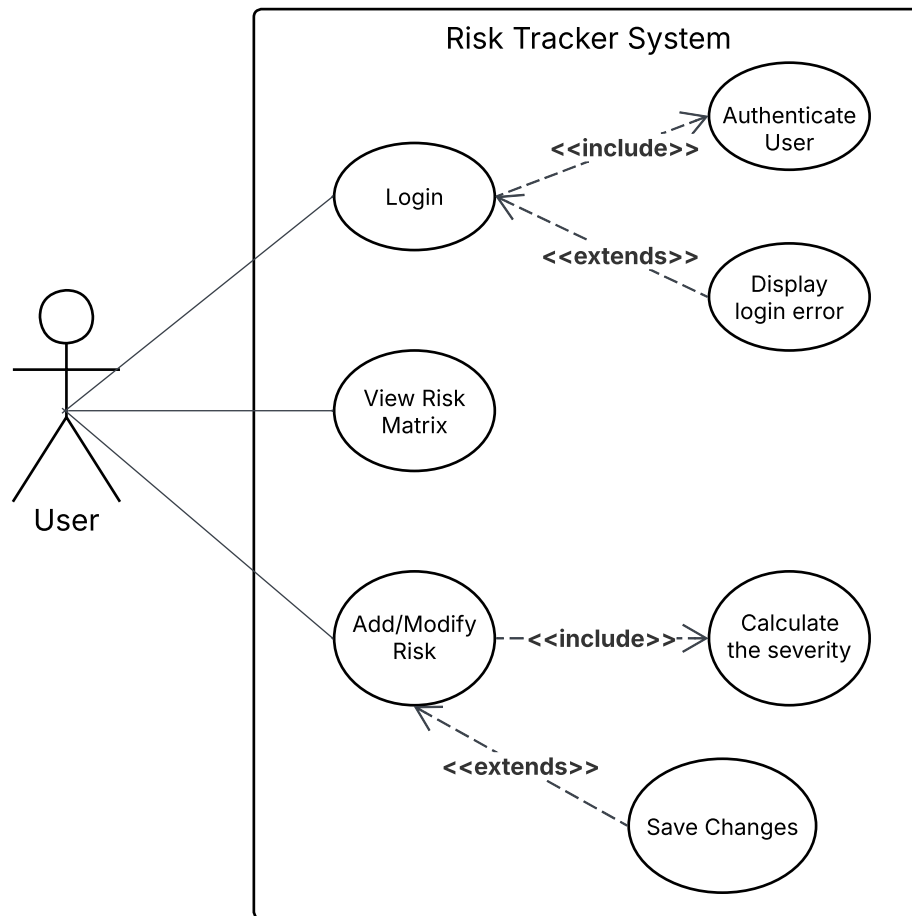


Figure 3.3: User Use Case

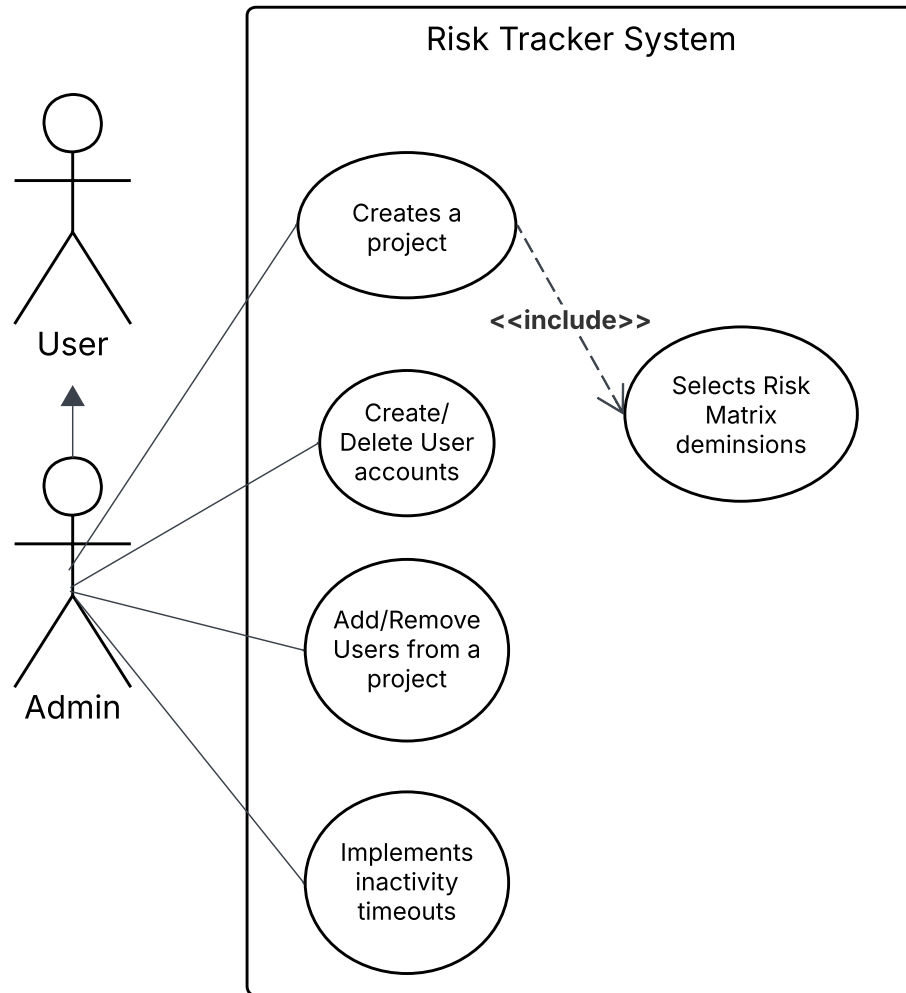


Figure 3.4: Admin Use Case

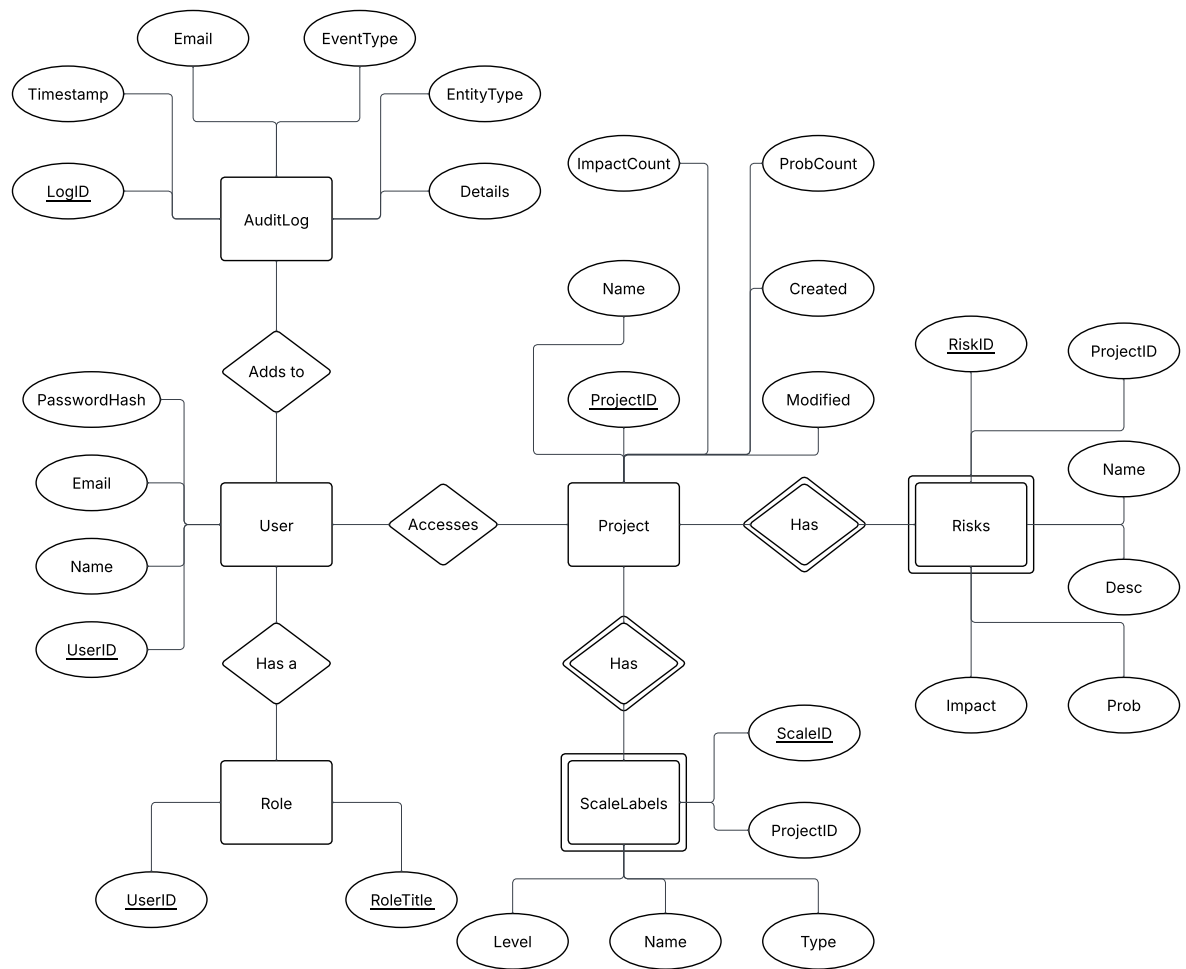


Figure 3.5: Database ERD

Each module interacts in a clear, step-by-step way, so data flows smoothly through the system.

3.3 Design Rationale

Why modular?

Dividing the system into modules makes it easier to understand, test, and update each part separately.

Why a database?

All data (users, risks, projects) must be stored reliably. A database keeps the data safe and organized.

4 DATA DESIGN

4.1 Data Description

The Software Risk Tracker employs a relational database model implemented using SQLite to ensure data persistence and integrity. Data is entered via the web interface, processed by the C++ Application Layer, and mapped to internal object structures (User, RiskItem, Project). These objects are then serialized and stored in the SQLite database files. The system utilizes a lightweight schema to minimize overhead while maintaining relationships between Users, Projects, and Risks.

4.2 Data Dictionary

Label:

Description: encapsulates all the information corresponding to a level label of a given project.

Attributes:

Name: String, the name given to identify the level.

Type: Character, identifies the type of label it is (impact or prob.)

Level: Unsigned integer, identifies the level of the label.

Matrix:

Description: contains the matrix's chosen dimensions.

Attributes:

Probability: Unsigned integer, the total number of probability levels.

Risk: Unsigned integer, the total number of risk levels.

Project:

Description: encapsulates all the information corresponding to the project.

Attributes:

Name: String, the name given to identify the project.

ImpactCount: Unsigned integer, the number of impact levels.

ProbCount: Unsigned integer, the number of probability levels.

Labels: List of Label objects, the level names associated with the project.

RiskItem:

Description: encapsulates all the information corresponding to the risk.

Attributes:

Name: String, the name given to identify the risk.

Desc: String, a short description of the risk.

Prob: Unsigned integer, the probability of the risk occurring.

Impact: Unsigned integer, the level of impact if the risk occurs.

UserInfo:

Description: groups identification information.

Attributes:

UserName: String, the username chosen to identify the account.

Email: String, the email linked to the account.

Password: String, the password related to the account.

5 COMPONENT DESIGN

5.1 Components

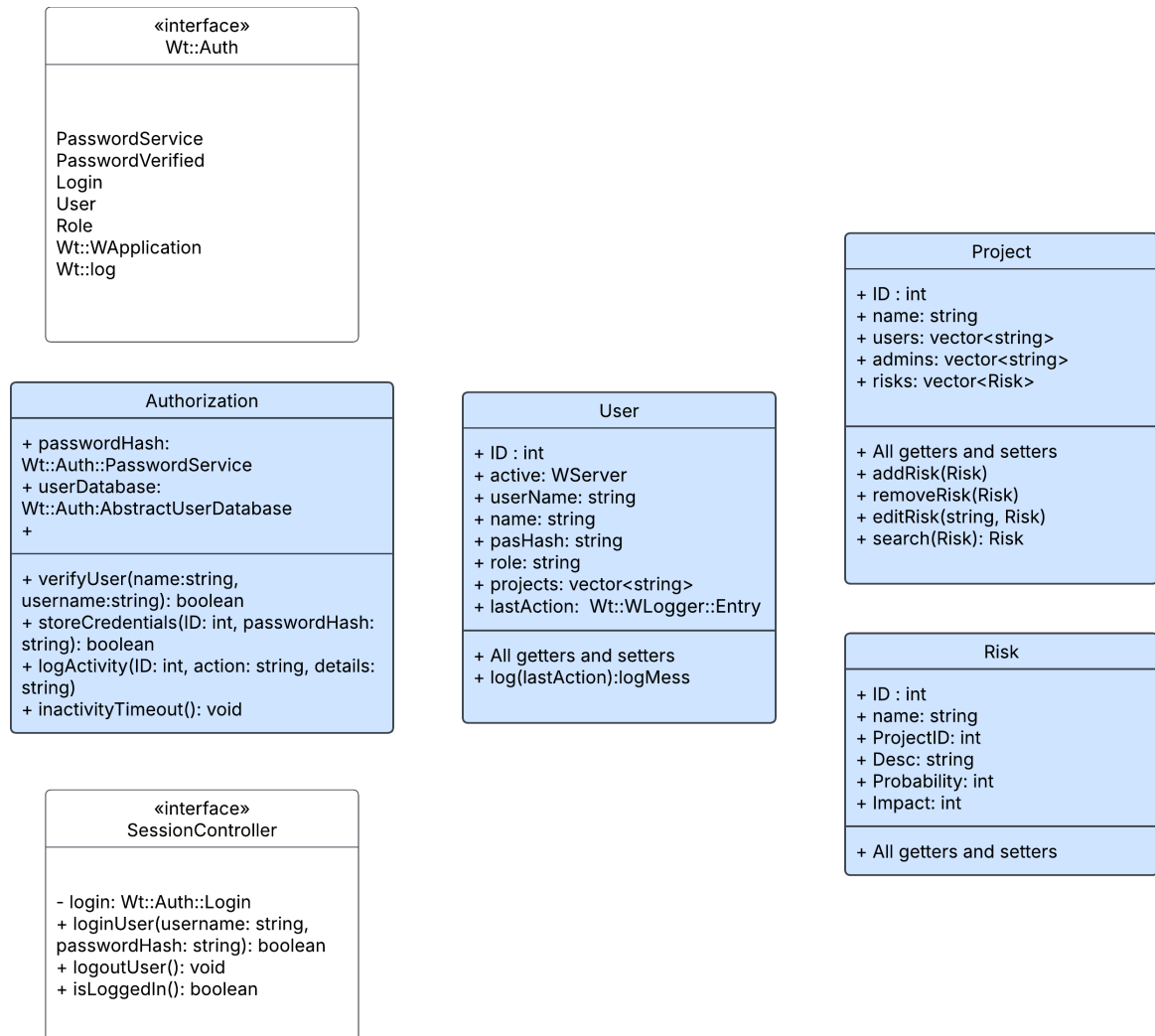


Figure 5.1: Overall Class Structuring

Login and Navigation for User Activity Diagram/Flow

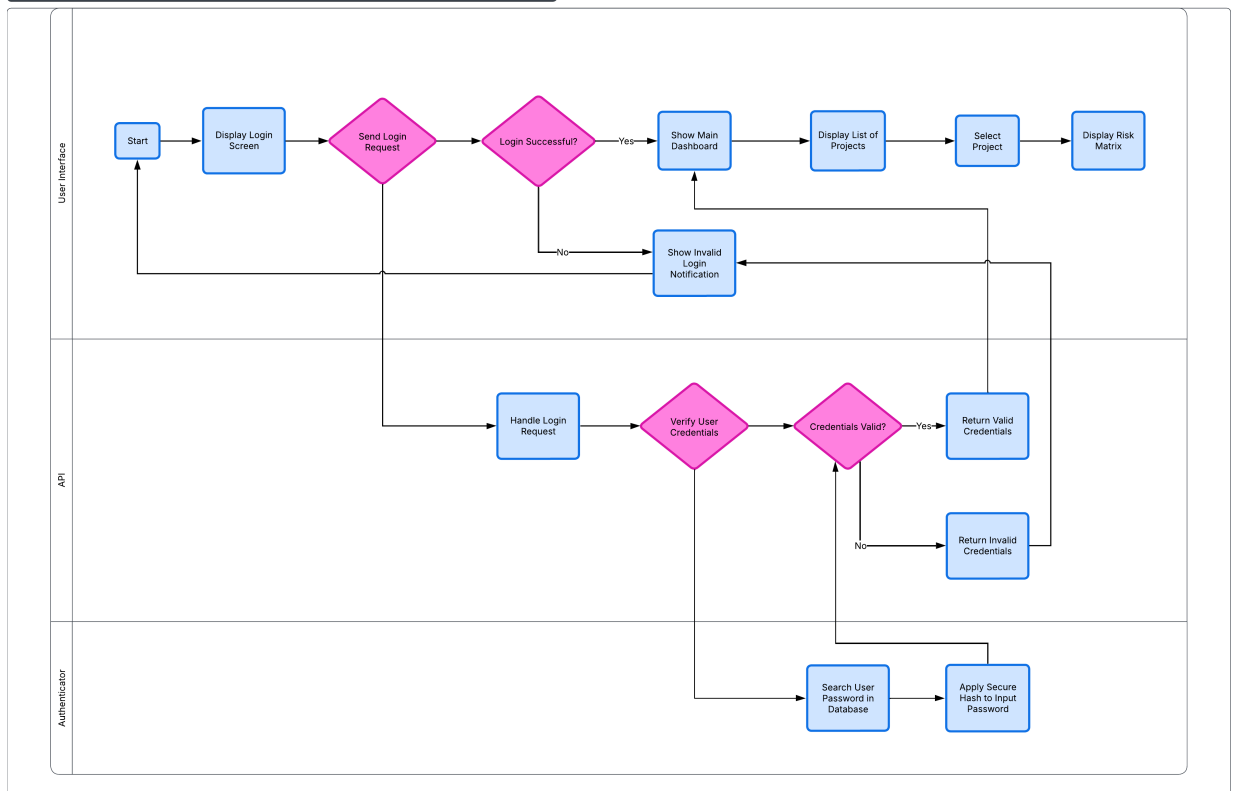


Figure 5.2: User Login/Navigation Control Flow

Add Risk Activity Diagram

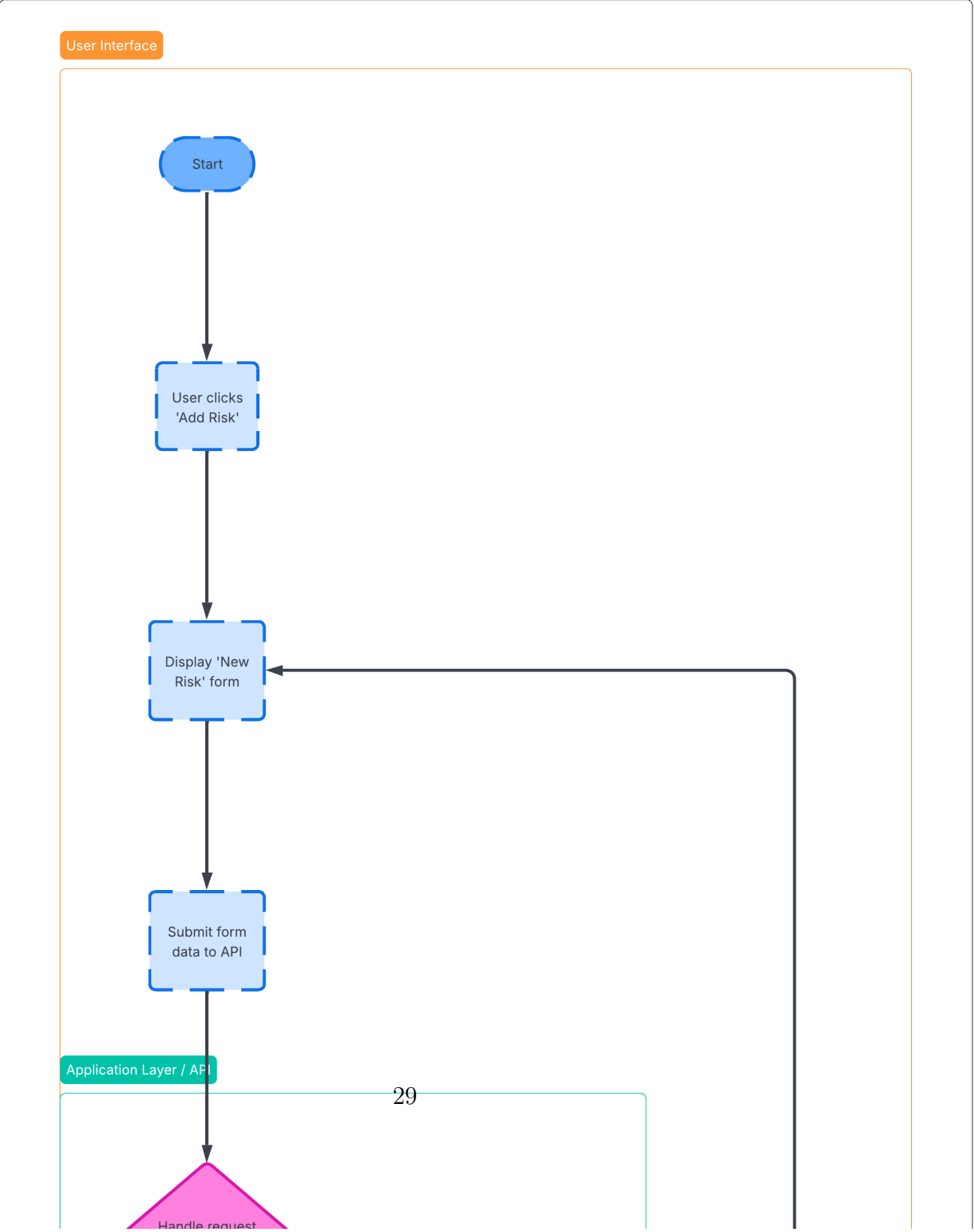


Figure 5.3: Add Risk Control Flow

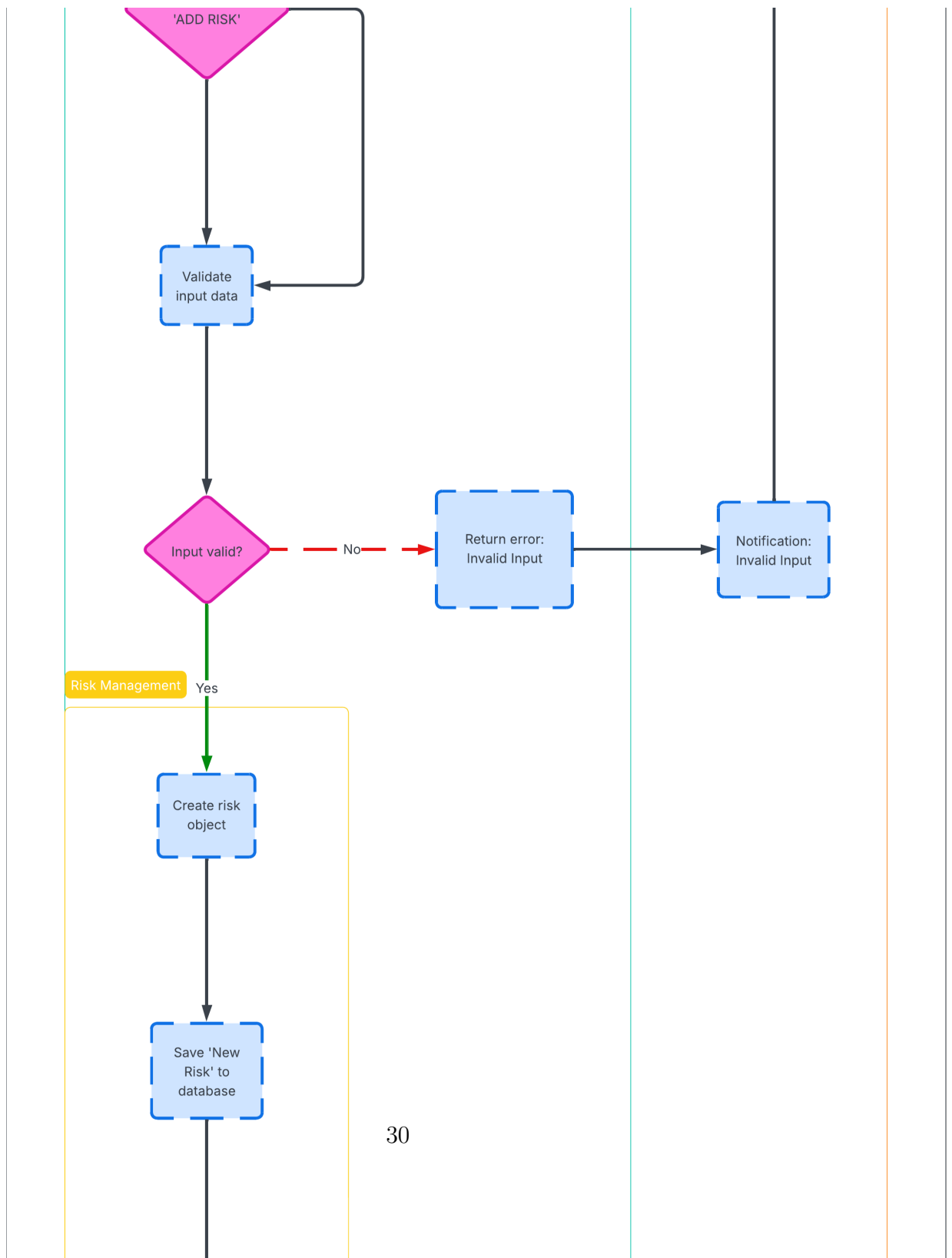


Figure 5.3: Add Risk Control Flow (Continued)

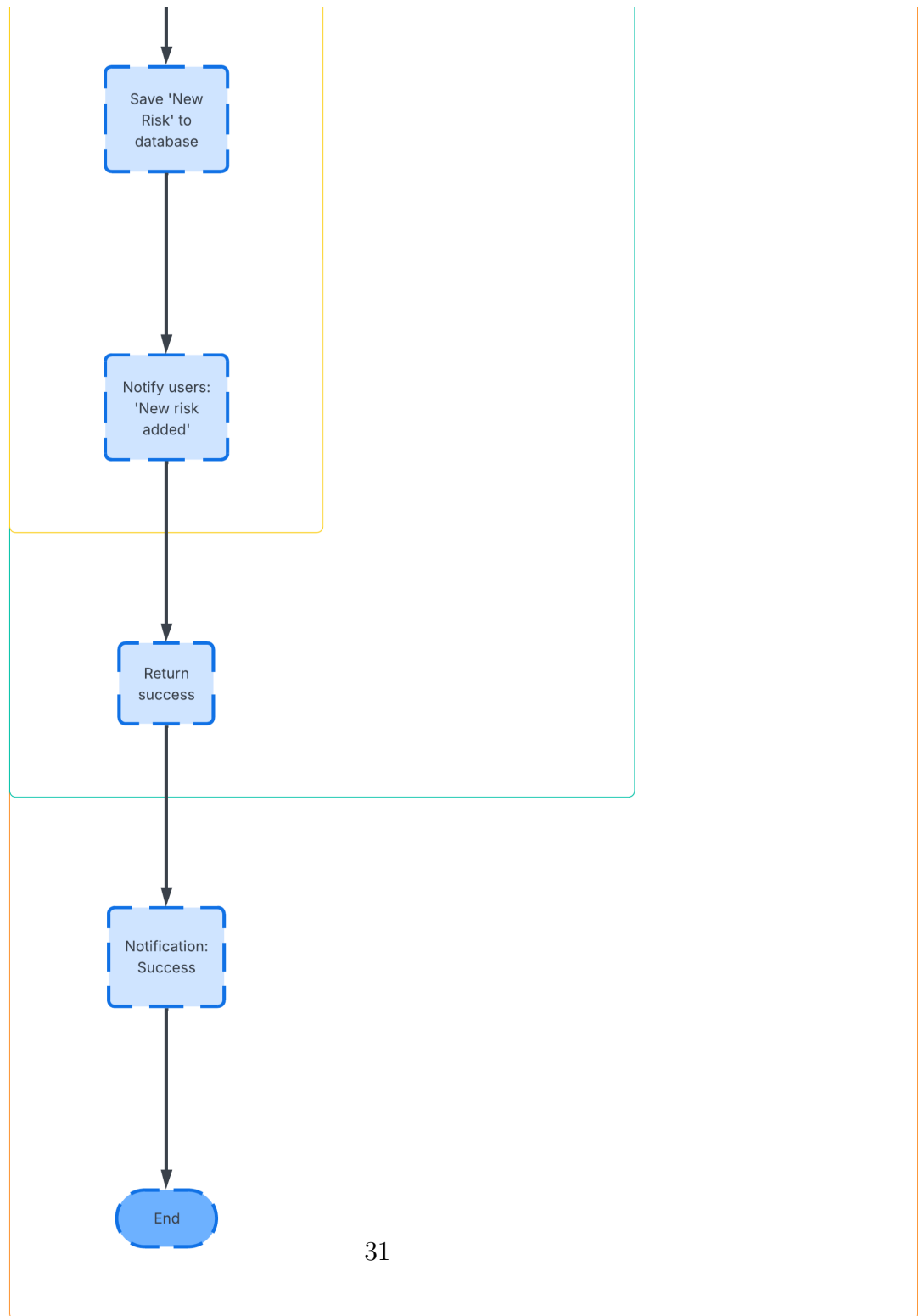


Figure 5.3: Add Risk Control Flow (Continued)

User Administration Activity Diagram

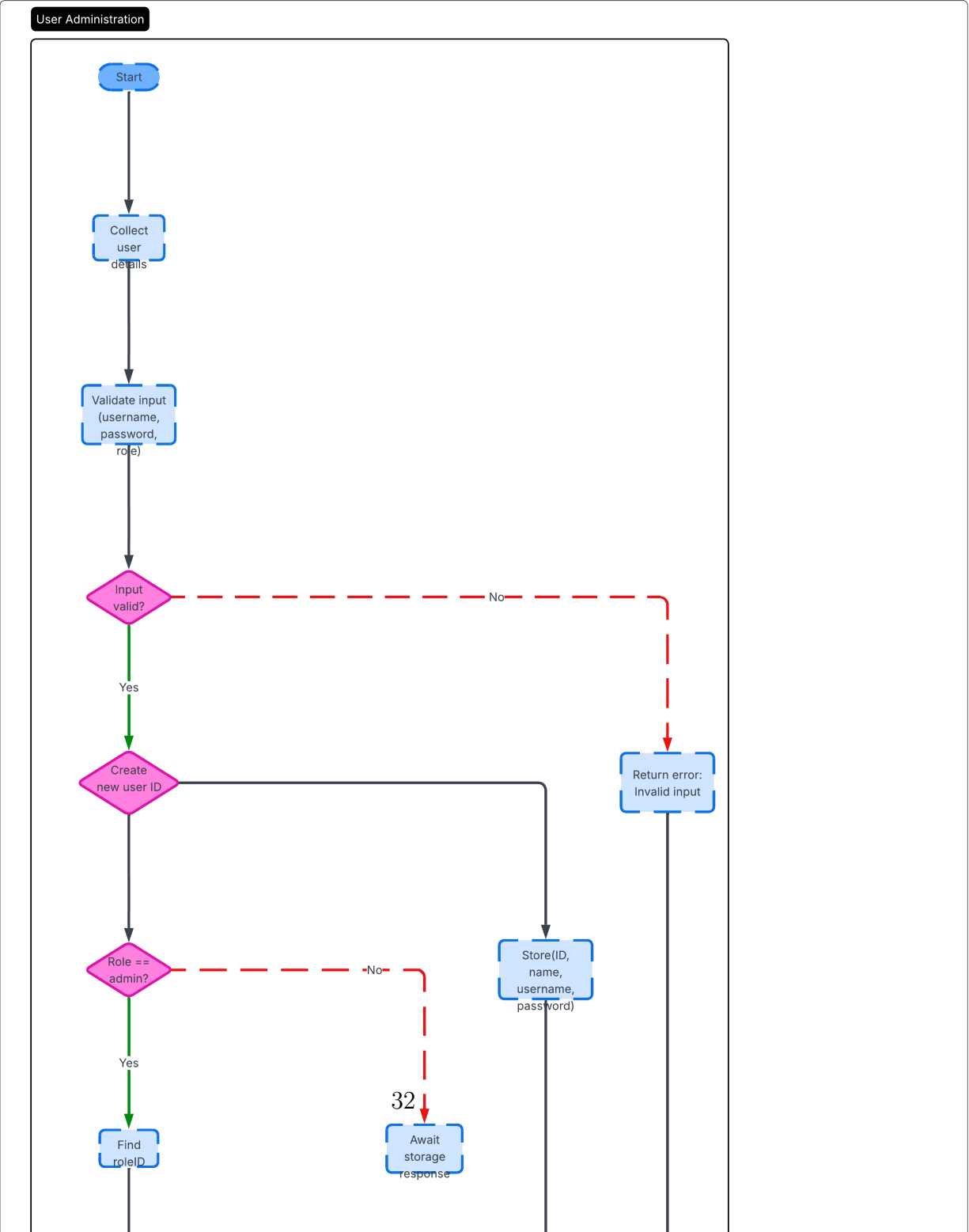


Figure 5.4: User Administration Control Flow

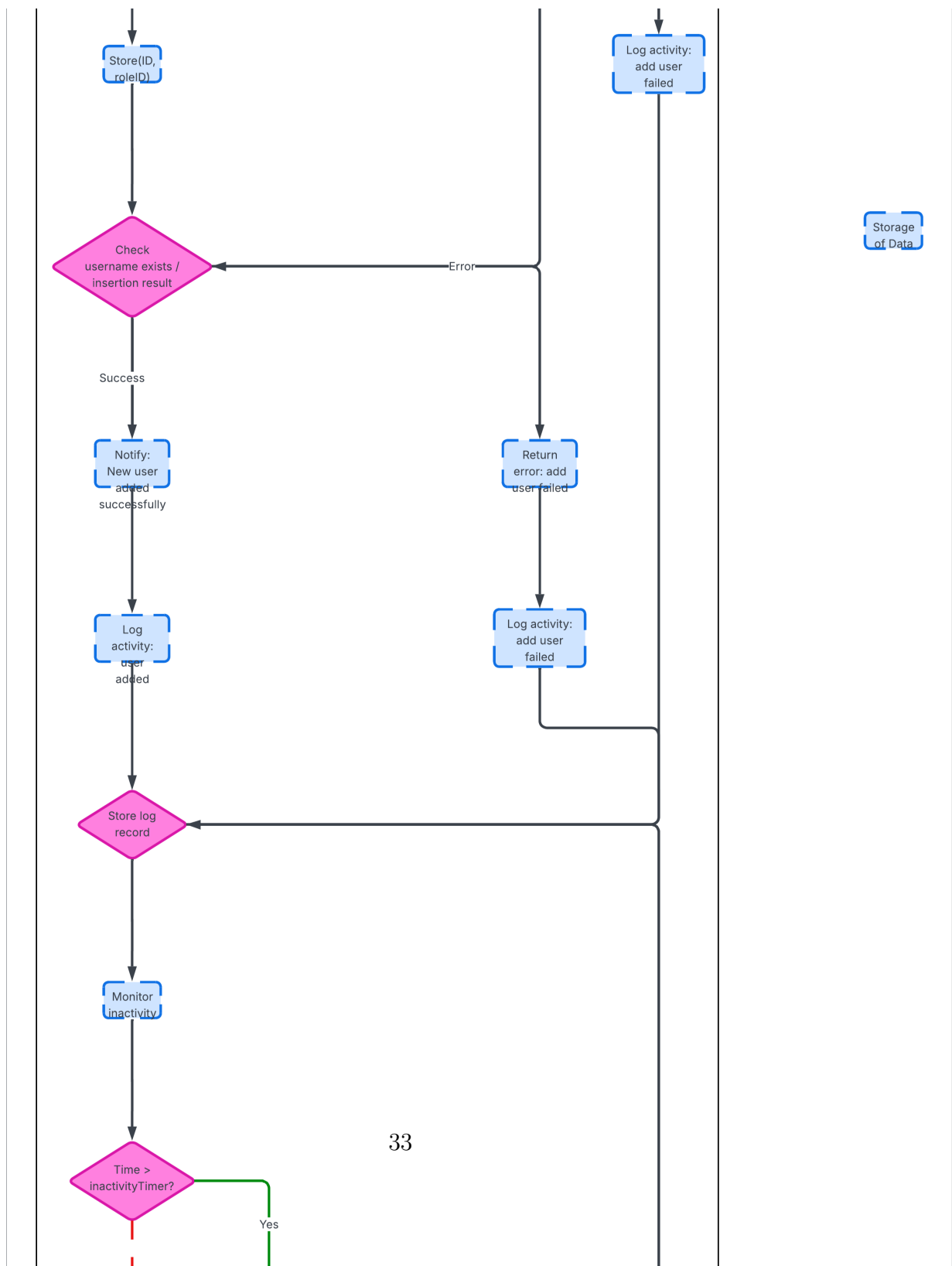


Figure 5.4: User Administration Control Flow (Continued)

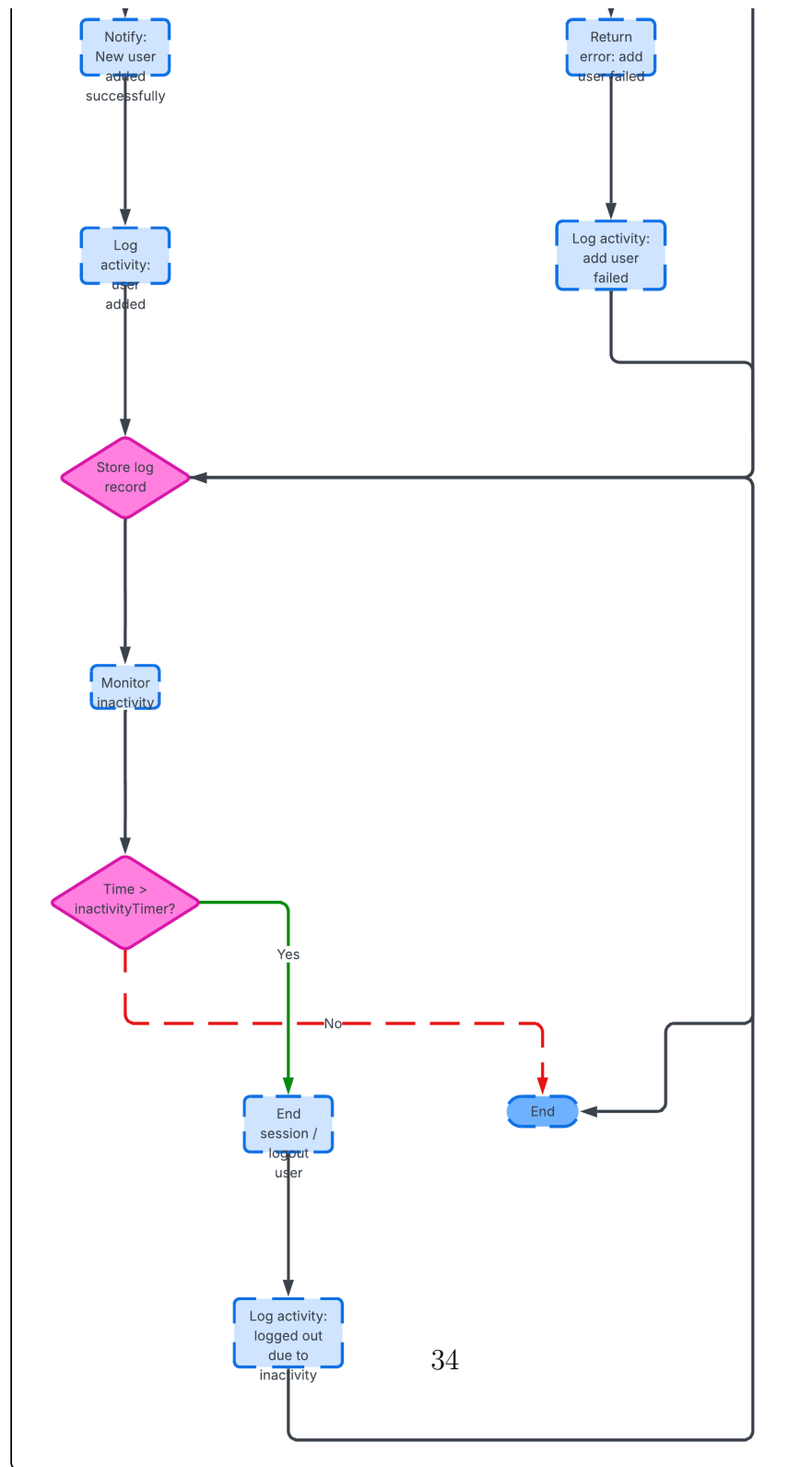


Figure 5.4: User Administration Control Flow (Continued)

5.1.1 User Interface (UI)

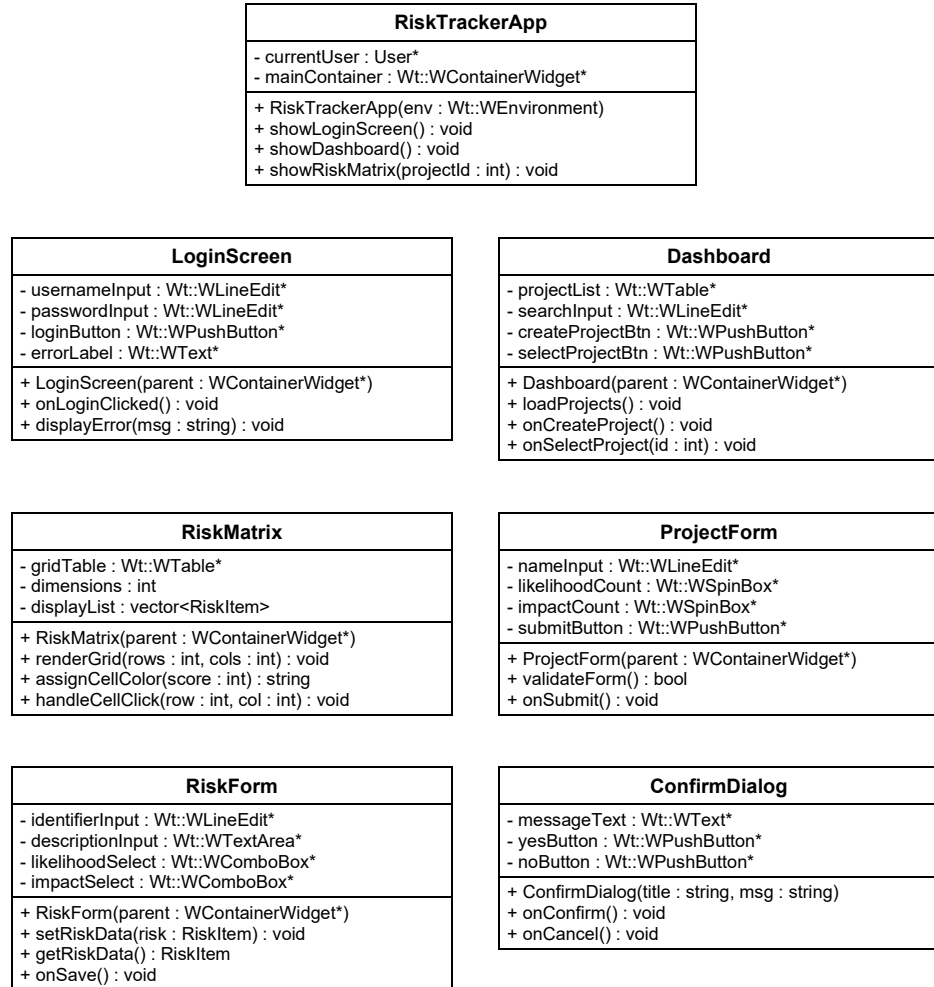


Figure 5.5: User Interface Class Diagram

Purpose:

The User Interface allows users to view risks, add new ones, and manage projects through a web browser.

Description:

- Displays lists of risks and details about each one.
- Provides forms for adding or editing risks.
- Sends user actions (“add risk” or “login”) to the backend API.
- Receives and displays feedback messages.

Pseudocode;

When user opens application:

Display login screen

If user logs in successfully:

 Show main dashboard

 Display list of all projects

 If user selects a project:

 Display the risk matrix

 When user clicks ”Add Risk”:

 Open form for new risk

 Send risk data to backend API

 When user clicks ”View Risk”:

 Request risk details from backend

 Display information on screen

5.1.2 Application Layer (API)

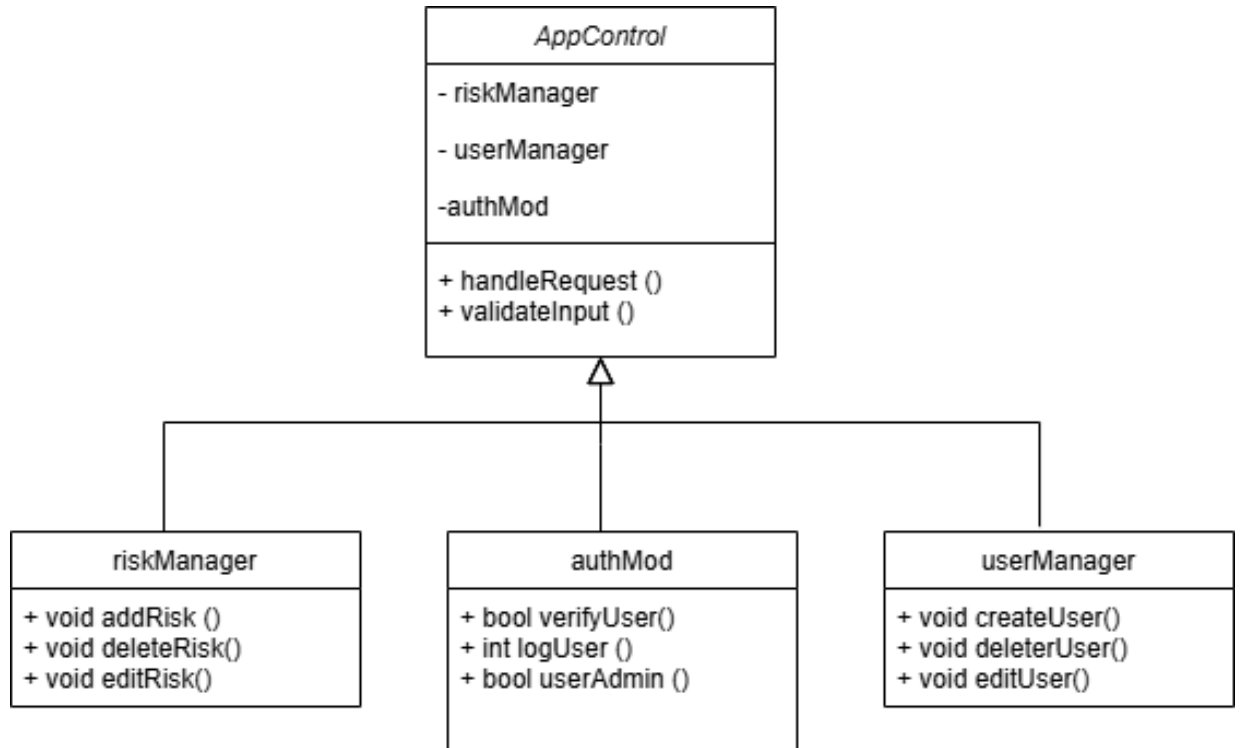


Figure 5.6: API Class Diagram

Purpose:

The Application Layer serves as the bridge between the User Interface and the backend logic. It receives requests from the browser, validates the input data to ensure it meets requirements, and routes the request to the appropriate manager (Risk or User).

Description:

- Listens for HTTP requests from the Witty web server.
- Validates user inputs (e.g., checks if risk probability is between 1-5).

- Routes valid requests to the Risk Management or User Management modules.
- Returns success or error messages back to the UI.

Pseudocode;

Function `handle_request(request_type, data)`:

 If `request_type` is "LOGIN":

 return `AuthModule.verify_user(data.username, data.password)`

 Else If `request_type` is "ADD_RISK":

 If `validate_input(data)` is True:

 return `RiskModule.add_risk(data)`

 Else:

 return "Error: Invalid Input"

Function `validate_input(data)`:

 If `data.probability < 0` OR `data.probability > 5`:

 Return False

 Return True

5.1.3 Authenticator

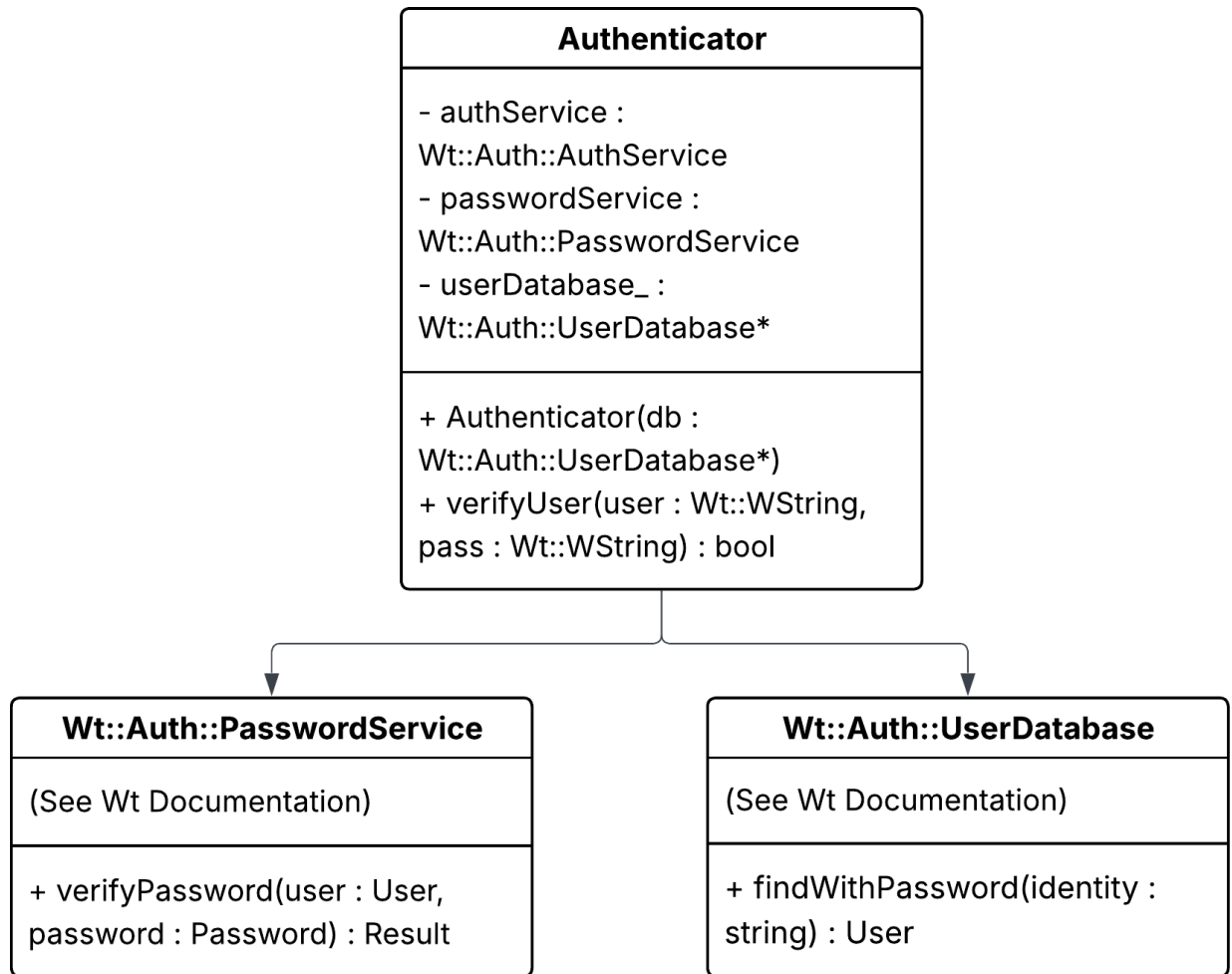


Figure 5.7: Authenticator Class Diagram

Purpose:

The Authenticator module handles system security by verifying user identities.

Description:

- Verifies user credentials (username and password) during login.
- Securely hashes passwords before comparing them.

Pseudocode:

Function verifyUser(username, inputPassword):

 userPassword = database.findPassword(username)

 hashedPassword = Apply secure_hash to inputPassword

 If userPassword == hashedPassword:

 Return "Valid Credentials"

 Return "Invalid Credentials"

5.1.4 User Administration

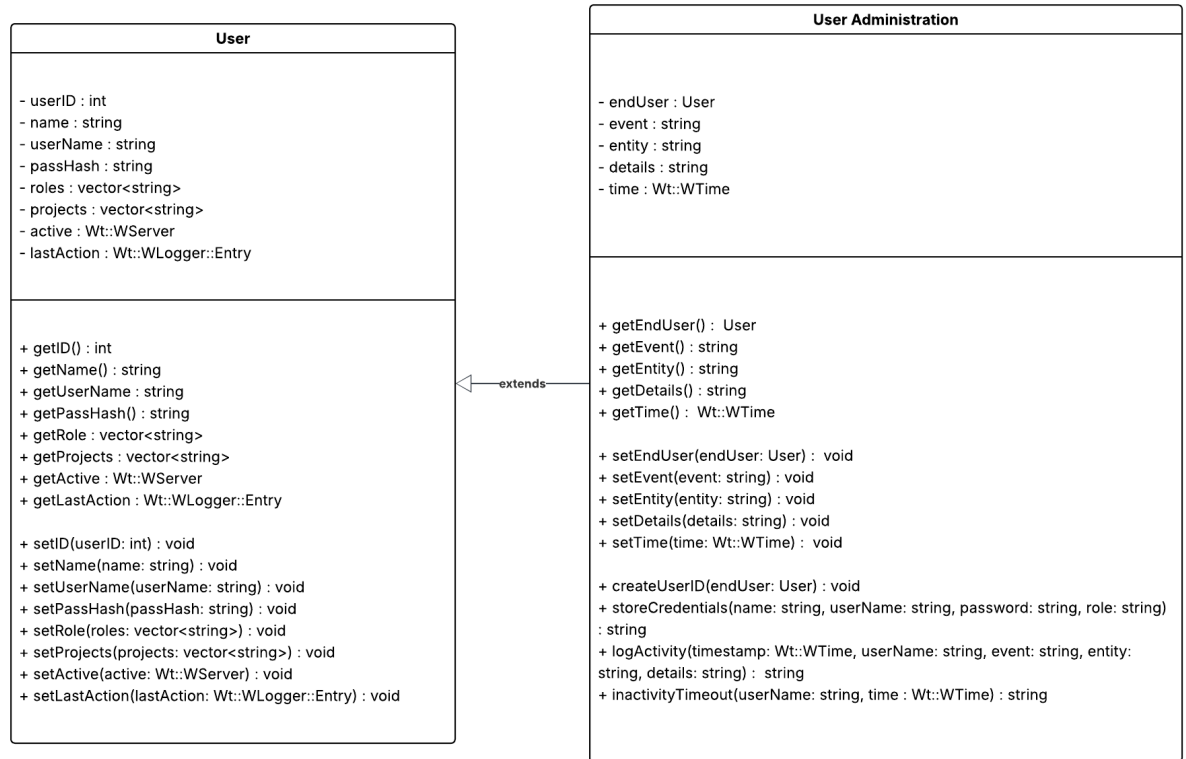


Figure 5.8: User Administration Diagram

Purpose:

The User Administration module manages user account information.

Description:

- Stores new users' information (username, password, etc.).
- Logs important user activities (login, logout, data changes) for security look over.
- Performs inactivity timeouts.

Pseudocode:

Function storeCredentials(name, userName, password,role):

 Create new userID

 database.store(userID, name, userName, password)

 If role == admin:

 roleID = database.find(role)

 database.store(ID, roleID)

 If database.find(userName) is true:

 Return "New user added successfully"

 Return "Error: new user could not be added"

Function logActivity(timestamp, userName, event, entity, details):

 database.store(timestamp, username, event, entity, details)

 Return Successfull

Function inactivityTimeout(userName, time):

 If time is greater than inactivityTimer:

 Logout the user

 Return "Logged out due to inactivity. Log back in."

5.1.5 Risk Management

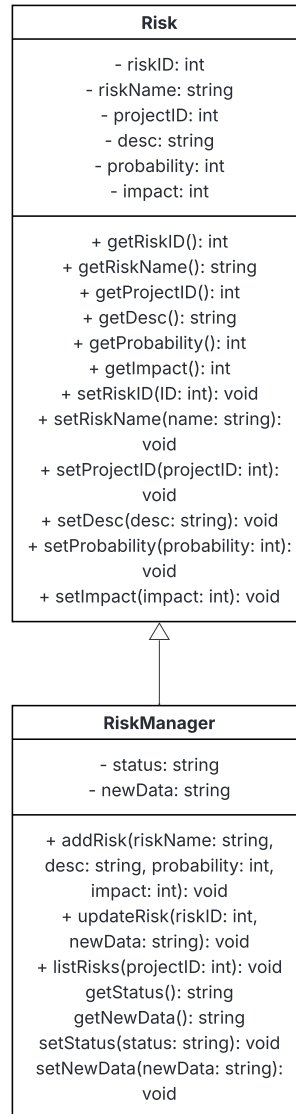


Figure 5.9: Risk Management Diagram

Purpose:

- Handles the creation, updating, and tracking of project risks.

Description:

- Adds new risks to the system.
- Updates the status, severity, or likelihood of existing risks.
- Removes risks when resolved.
- Provides lists and summaries of risks for a project.

Pseudocode;

```
Function addRisk(title, description, severity, likelihood):  
newRisk = createRiskObject(title, description, severity, likelihood)  
database.save(newRisk)  
notifyUsers("New risk added: " + title)  
Return "Risk created successfully"
```

```
Function updateRisk(riskID, newData):
```

```
risk = database.findRisk(riskID)
```

```
If risk exists:
```

```
Update risk fields with new_data
```

```
database.save(risk)
```

```
Return "Risk updated"
```

```
Else:
```

```
Return "Risk not found"
```

```
Function listRisks(projectID):
```

```
risks = database.getRisks(projectID)
```

```
Return risks
```


5.1.6 Storage of Data

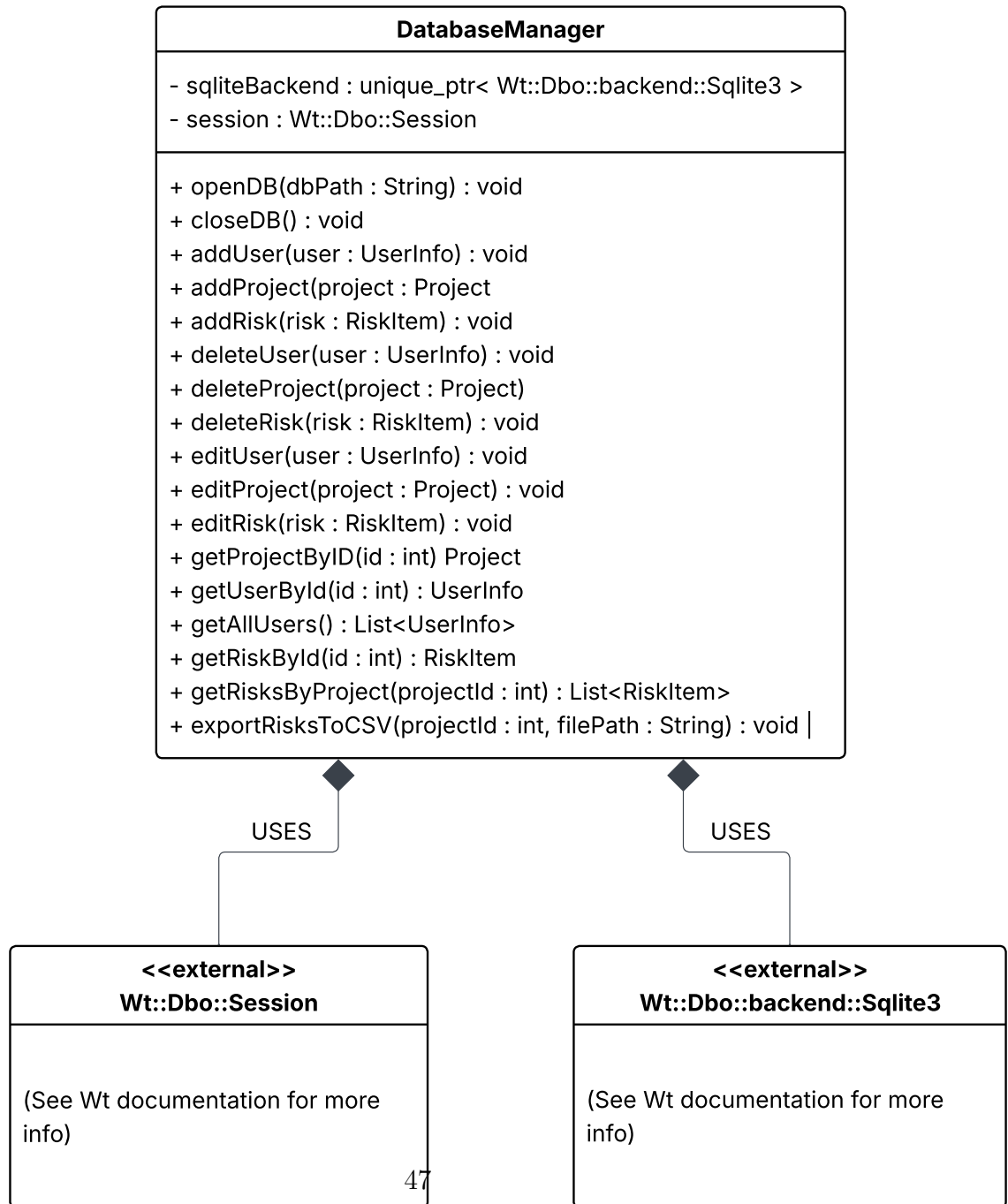


Figure 5.10: Storage of Data Class Diagram

Purpose:

The Storage of Data module, shown in the diagram 5.9 above, acts as the interface between the application logic and the SQLite database, ensuring data is securely saved, retrieved, and exported.

Description:

- Manages the connection to the local SQLite database file.
- Executes SQL queries to insert new Users or Risks into their respective tables.
- Retrieves data rows when requested by other modules.
- Exports project risk data from the database to CSV files.

6 HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

The user will be presented with a login screen where they will enter in their username and password. Then they will be given the option to start a new project or to view past projects. See fig 6.2. If the user selects the new project option they will be able to choose the risk matrix's dimensions and then they will be able to see a matrix you can interact with that is color-coded ranging from green to red. See fig 6.1. The two axes, X and Y will correlate with the likelihood that a risk is to occur and the severity of that risk. See fig 6.1. The user can interact with the matrix by clicking on one of the squares within the matrix and then may type the risk name and its description. That square will then display the text the user entered. It will save this information for future reference by clicking on the save button. There will also be an edit button for future modifications to the risk matrix shown in fig 6.2.

6.2 Screen Designs

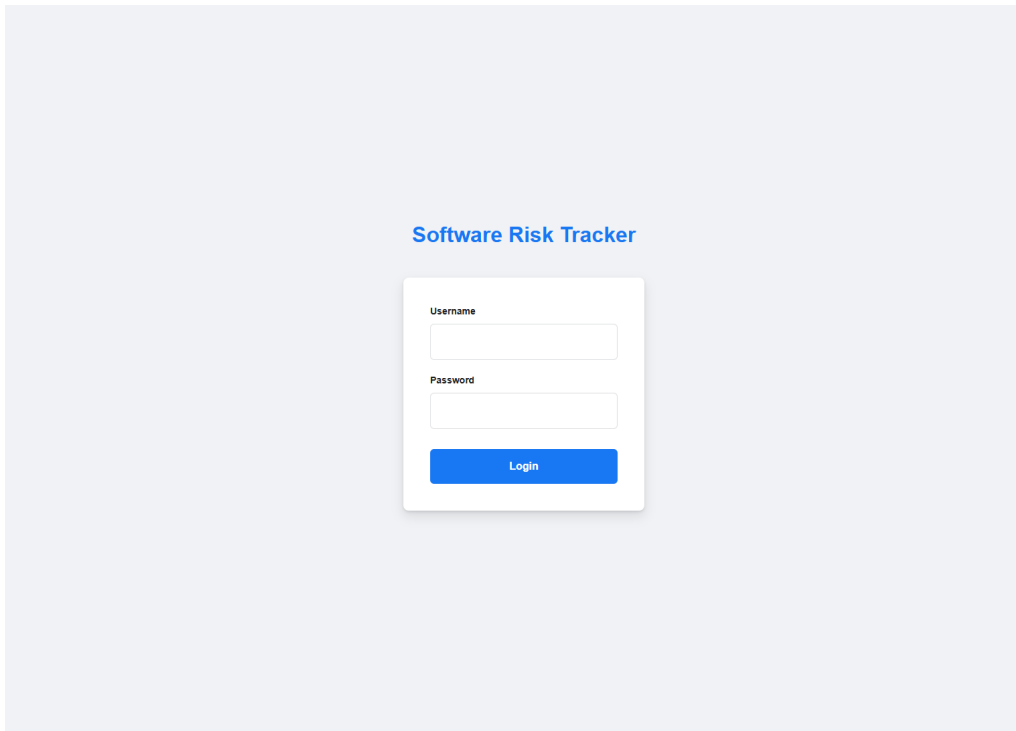


Figure 6.1: Risk Tracker Login

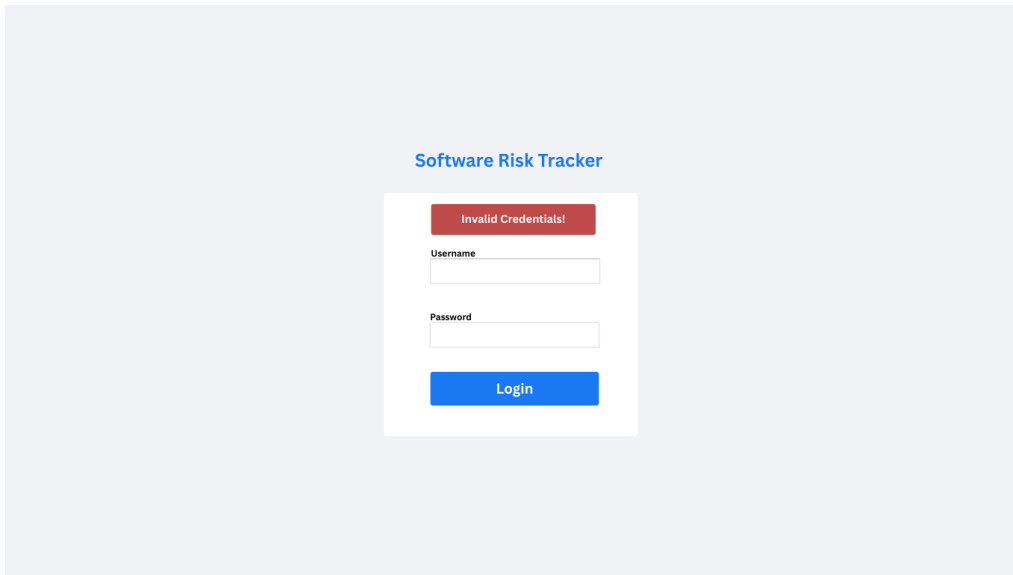


Figure 6.2: Risk Tracker Login Fail

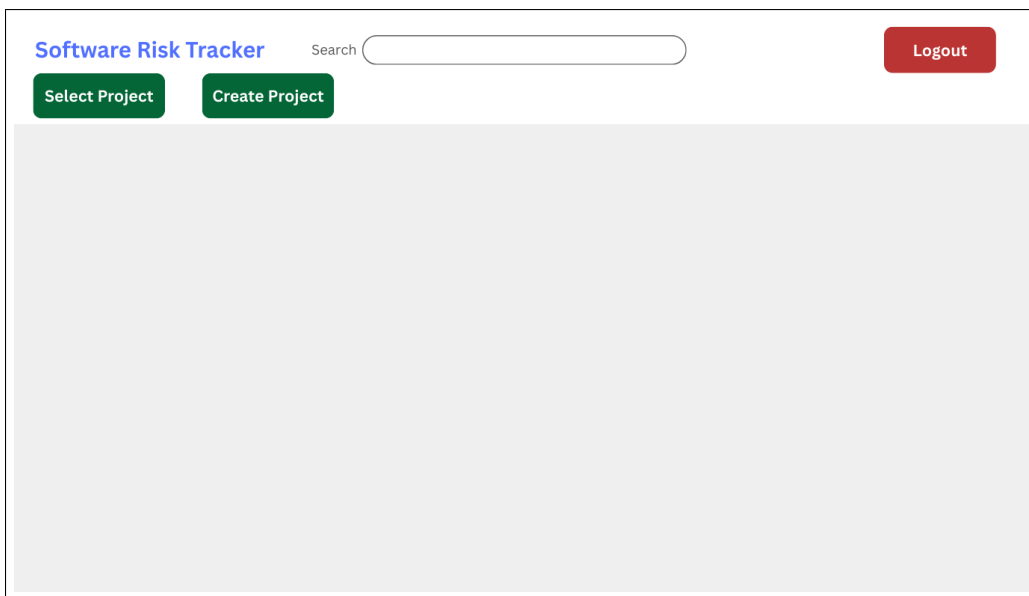


Figure 6.3: Non-Admin User Dashboard

Software Risk tracker

Create Project

Project Name

Number of Risk Likeliness

1100

Number of Risk Matrix

1100

Create Project

Figure 6.4: Create Project

Software Risk tracker

Edit Project

Project Name

Project-1

Number of Risk Likeliness

10

Number of Risk Impacts

10

Save Edit

Figure 6.5: Edit project

Software Risk tracker

Project view

Add Risks

Project-1

Risk Identifier

Risk Likelihood

Risk Description

Risk Impact

Add Risk

Figure 6.6: Add new risk to Matrix

Software Risk tracker

Project view

Add Risks

Edit Risks

Project-1

Risk Identifier

Failure in security

Risk Likelihood

1+

Risk Description

Leaking of information

Risk Impact

10+

Save Edit

Figure 6.7: edit current risk

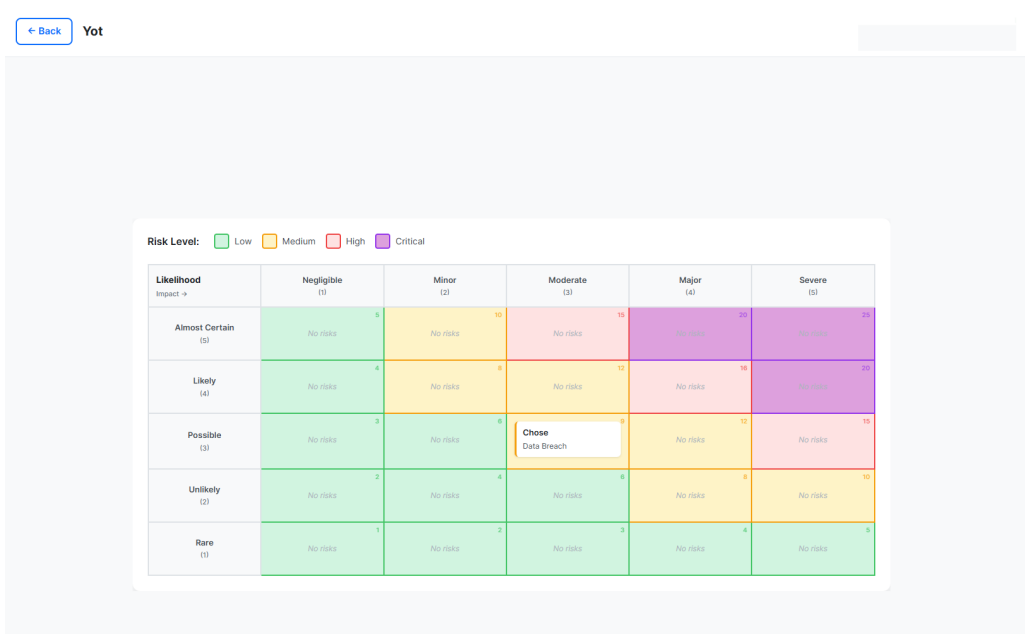


Figure 6.8: Matrix display

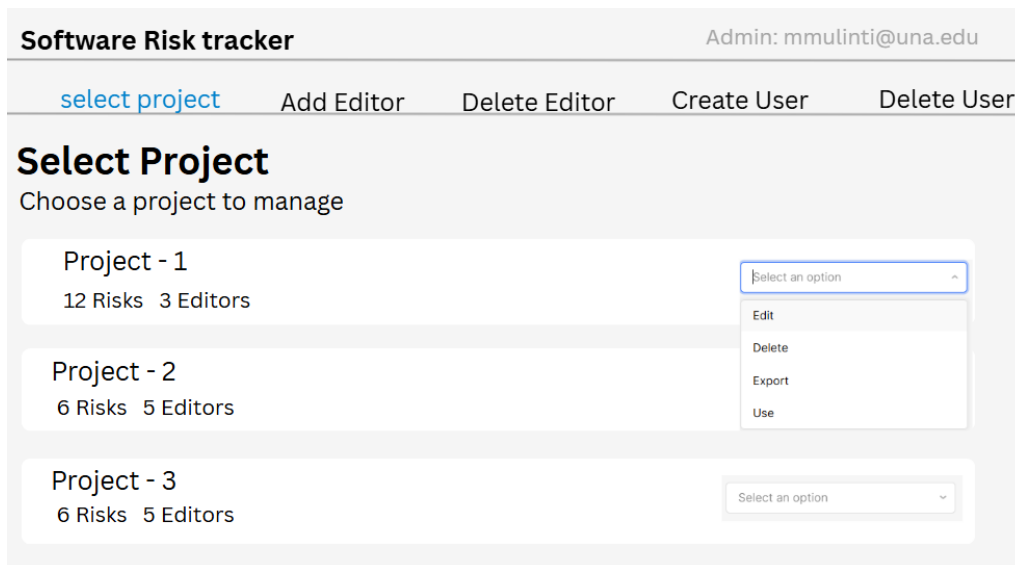


Figure 6.9: Admin dashboard - select project

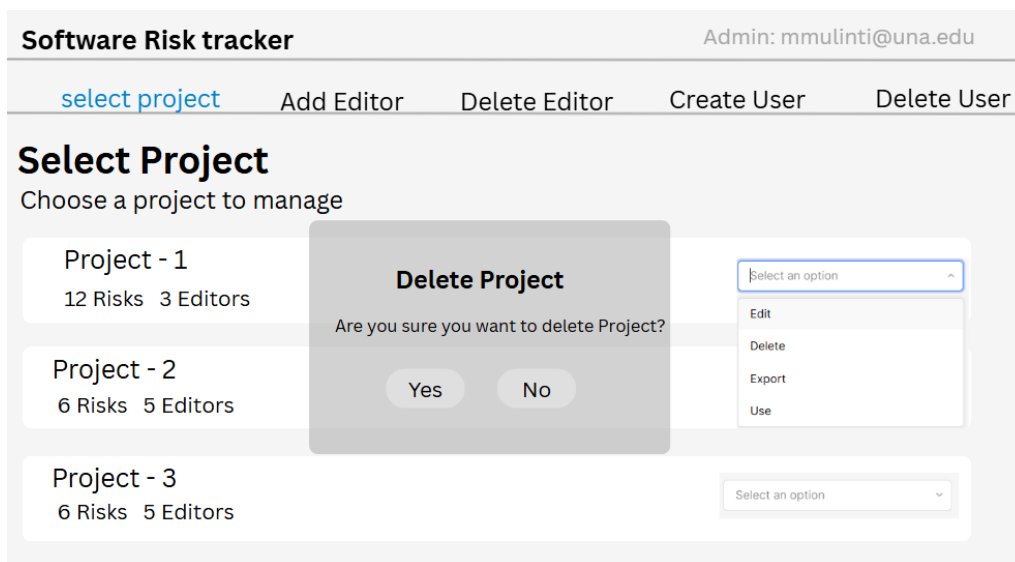


Figure 6.10: confirm delete Project

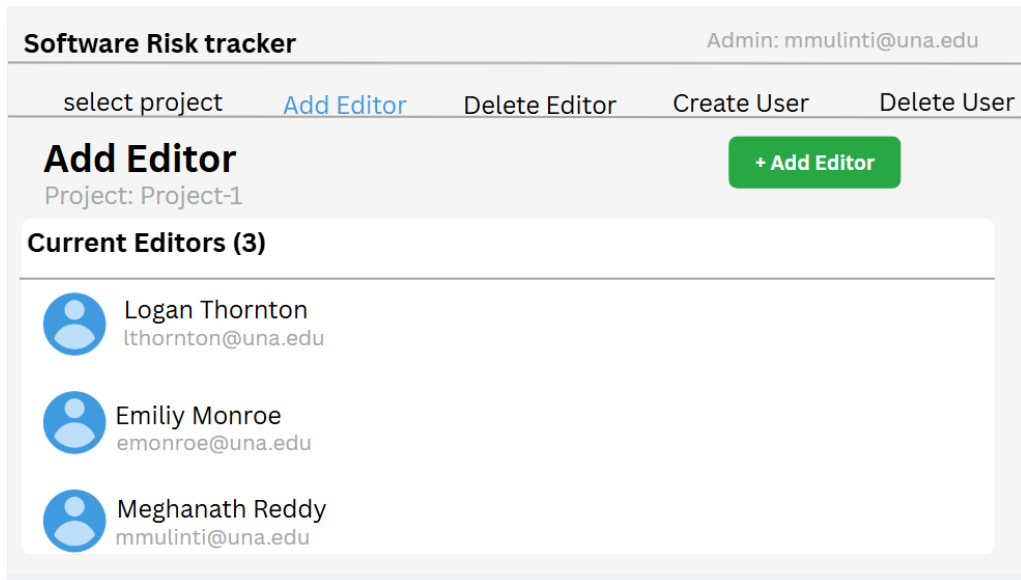


Figure 6.11: Add Editor

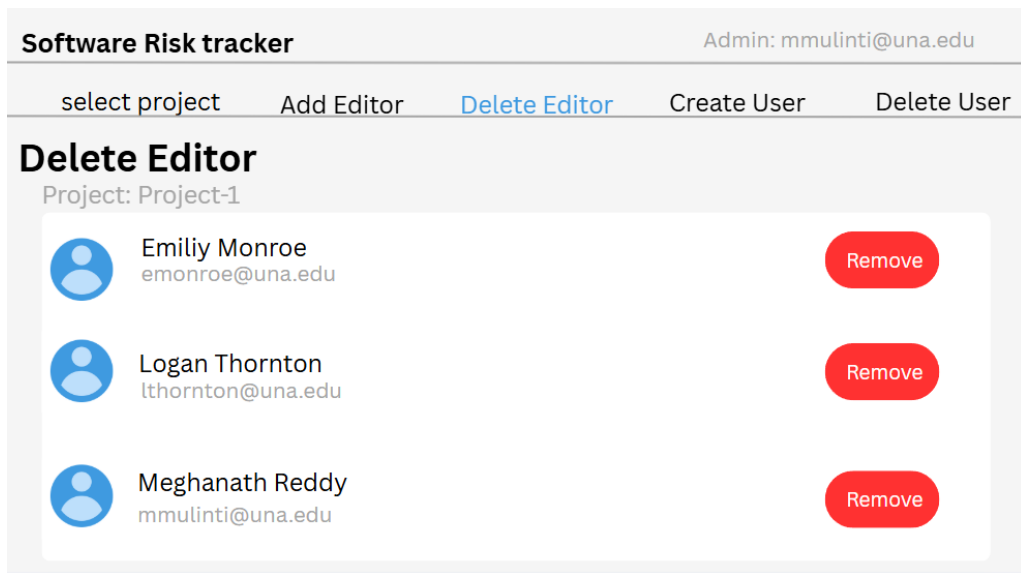


Figure 6.12: Remove Editor

Software Risk tracker
Admin: mmulinti@una.edu

select project
Add Editor
Delete Editor
Create User
Delete User

Delete Editor

Project: Project-1



Emiliy Monroe
emonroe@una.edu



Logan Thornton
lthornton@una.edu



Meghanath Reddy
mmulinti@una.edu

Delete Editor

Are you sure you want to remove editor from this Project?

Yes
No

Remove
Remove
Remove

Figure 6.13: Confirm remove editor

Software Risk tracker
Admin: mmulinti@una.edu

select project
Add Editor
Delete Editor
Create User
Delete User

Create User

Add new user

Full Name

Email

Password

Confirm Password

Role

-- Select Role --

Create User

Figure 6.14: Create User

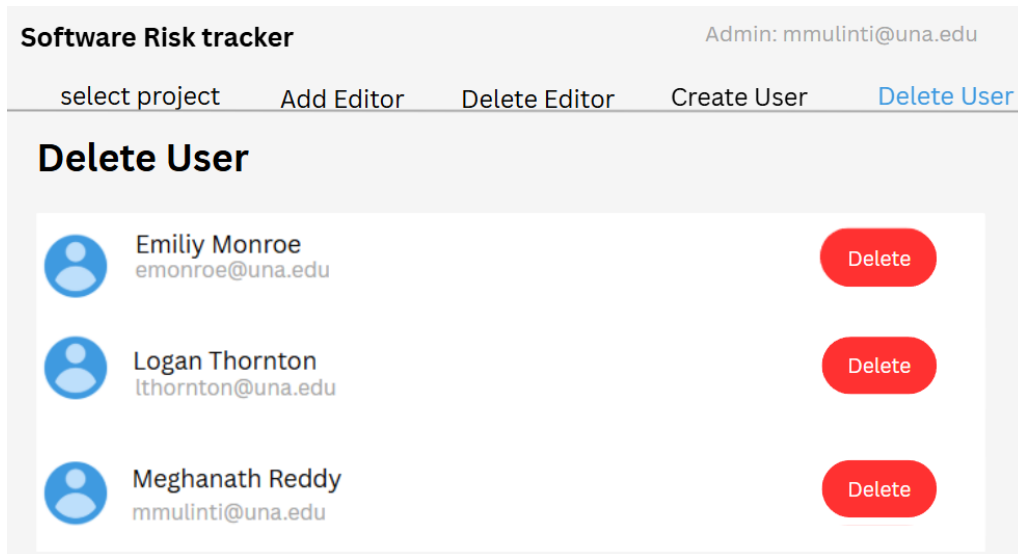


Figure 6.15: Delete User

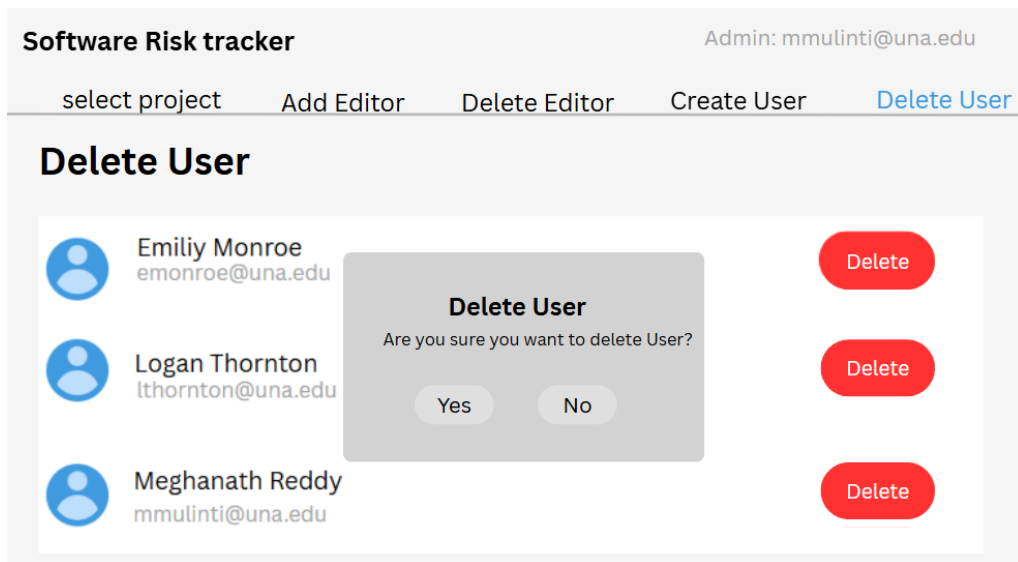


Figure 6.16: Confirm delete user

7 REQUIREMENTS TRACEABILITY MATRIX

Req. ID	Description	Component
PERF-1	The system shall support up to 20 concurrent users while maintaining an average response time for all user actions under 2 seconds.	API Module
PERF-2	The response time to create, edit, or delete a risk shall be less than 2 seconds.	API Module, RM Module
PERF-3	The response time to save user input for a risk (e.g., identifier, description, likelihood) shall be less than 1 second.	API Module
PERF-4	The response time for loading any new page (e.g., dashboard after login, home screen) shall be less than 5 seconds.	API Module, UI Module
PERF-5	The response time to generate a complete risk matrix shall be less than 4 seconds.	API Module, UI Module
PERF-6	The response time to update the risk matrix after a risk is created, edited, or deleted shall be less than 2 seconds.	API Module, RM Module
<i>Continued on next page</i>		

Req. ID	Description	Component
PERF-7	The time it takes the system to export risk data to CSV shall be less than 2 seconds.	SoD Module
SAFE-1	The system shall require a user to confirm any deletion action.	UI Module
SAFE-2	The system shall automatically log out any user session that has been inactive for 15 minutes.	AUM Module
SECU-1	The system shall require successful user authentication before granting access to any application functionality.	AUM Module
SECU-2	User data shall be securely stored against data loss.	SoD Module
SECU-3	Passwords shall be stored with a secure hashing algorithm.	SoD Module
SECU-4	User activity shall be recorded in a log for the following actions: login, action performed (create, delete, edit), and CSV file exported.	AUM Module
SECU-5	If a user initiates a password reset, the system shall send a time-limited, single-use code to the user's registered email address to facilitate the reset.	AUM Module
QUAL-1	The system shall follow the Mozilla coding style.	Mozilla Coding Style
QUAL-2	The system shall follow the SEI secure coding style.	Cppcheck/Static Analysis
QUAL-3	The system shall meet the accessibility standards provided by WCAG 2.1.	UI Module
<i>Continued on next page</i>		

Req. ID	Description	Component
FUNC-1	The system shall allow users to create a new project by defining a project name	UI Module, API Module
FUNC-2	The system shall require user authentication (username and password) before granting access to project data	AUM Module
FUNC-3	The system shall allow users to input risk data including a unique Risk Identifier, Description, Likelihood, and Impact.	UI Module, RM Module
FUNC-4	The system shall allow the user to define the dimensions (number of rows and columns) of the risk matrix during project creation	UI Module, API Module
FUNC-5	The system shall store all input data and provide a feature to export the project data as a CSV file	SoD Module
FUNC-6	The system shall display a visual risk matrix table, dynamically color-coded from green to red based on the calculated risk level (Likelihood $\times$ Impact)	UI Module, API Module
FUNC-7	Regular users can access and edit the project information	AUM Module
FUNC-8	Admin users can delete projects, add users to projects, remove users from projects, edit projects, and create and delete users	AUM Module
FUNC-9	The software will comply with WCAG Guidelines.	UI Module

Table 7.1: Nonfunctional Requirements and their Corresponding Components